

API Reference — Aveno Marketplace

This is the detailed companion to [API_CONTRACT.md](#) . That file is the *contract* (route, auth, one-line note) the Flutter app is built against — this file is the *manual*: what each endpoint actually does, exactly what to send it, and exactly what it sends back, with real example payloads.

Every endpoint below has been exercised against a running instance of this API (both directly with curl/PHP and via the automated test suite in `tests/Feature/`) — the shapes here are observed, not guessed.

Conventions used throughout

- **Base URL (local dev):** `http://localhost:8000/api`
- **Auth header:** `Authorization: Bearer <token>` (Laravel Sanctum personal access token, obtained from `/auth/register` or `/auth/login`). No cookies, no CSRF.
- **Always send** `Accept: application/json` .
- **List endpoints** that are paginated return Laravel's default paginator shape:

```
{
  "data": [ ... ],
  "links": { "first": "...", "last": "...", "prev": null, "next": null },
  "meta": { "current_page": 1, "from": 1, "last_page": 1, "per_page": 20, "to": 2, "total": 2,
}
```

A few admin list endpoints (noted below) return the **full unpaginated set** instead — same `{"data": [...]}` wrapper, but no `links` / `meta` .

- **Single-item endpoints** return `{"data": { ... } }` (via a Laravel API Resource), except a handful of legacy actions that return a plain object or a `{"message": "..."}` acknowledgement — noted per endpoint.
- **Validation errors** → 422 with `{"message": "...", "errors": {"field": ["reason"]}}` .
- **Unauthenticated** → 401 with `{"message": "Unauthenticated."}` .
- **Forbidden** (wrong role / not the resource owner) → 403 with `{"message": "..."}` .
- **Not found** → 404 with `{"message": "..."}` (also used deliberately for ads that exist but aren't `approved` yet, to a non-owner — see Ads below).
- A `UserResource` (used inside many responses below) always has this shape:

```
{
  "id": 3, "name": "Test User", "phone": "0999111222", "email": null,
  "avatar_url": null, "rating_average": 0, "rating_count": 0,
  "is_banned": false, "created_at": "2026-09-03T19:13:45.000000Z"
}
```

(`governorate` / `city` appear here too, only when the caller eager-loaded them — they don't on any endpoint below.)

Auth

POST /auth/register

Auth: none. **Does:** Creates a new user account and immediately logs them in (returns a usable token — no separate login step needed after registering). **Input (JSON body):**

Field	Required	Notes
name	yes	string, max 100
phone	yes	string, must be unique — this is the login identifier
password	yes	string, min 8
email	no	must be a valid, unique email if given
governorate_id	no	must exist in <code>governorates</code>
city_id	no	must exist in <code>cities</code>

Output — 201 :

```
{
  "user": { "id": 3, "name": "Test User", "phone": "0999111222", "email": null, "avatar_url": null, "token": "4|JuyvPN0mHQqJir2X1y344aBi0wmr2R3ch7jGWf9K8775e271"
}
```

Errors: 422 if phone / email already taken or any rule fails.

POST /auth/login

Auth: none. **Does:** Authenticates by phone + password, returns a fresh token. Blocks banned users. **Input (JSON body):** phone (required), password (required). **Output — 200 :** same shape as register ({ "user": ..., "token": "..."}). **Errors:**

- 422 { "message": "بيانات الدخول غير صحيحة" } — wrong phone or password.
- 403 { "message": "تم إيقاف هذا الحساب" } — account is banned (is_banned = true).

POST /auth/logout

Auth: required. **Does:** Revokes the token used to make *this* request (other tokens/sessions for the same user, if any, stay valid). **Input:** none. **Output — 200 :** { "message": "تم تسجيل الخروج" }.

GET /auth/me

Auth: required. **Does:** Returns the caller's own profile — the standard way to check "am I still logged in" and to read `rating_average` / `rating_count` after a rating event. **Input:** none. **Output — 200 :** a single `UserResource` under `data` .

Categories

GET /categories

Auth: none. **Does:** Top-level (no parent), active categories only, with their immediate children — this is what feeds the home-screen category grid. **Input:** none. **Output — 200 :**

```
{
  "data": [
    { "id": 1, "name_ar": "سيارات", "name_en": null, "slug": "cars", "icon_url": null, "is_active": true }
  ]
}
```

(No `attributes` key here — that only loads on the single-category endpoint below. No pagination wrapper — this returns the full active top-level list every time.)

GET /categories/{id}

Auth: none. **Does:** One category with its dynamic spec fields (`attributes`) and its children — this is what renders the "post an ad" form once a category is picked, so the Flutter app knows which fields to show and how to validate them client-side. **Input:** `{id}` path param. **Output — 200 :**

```
{
  "data": {
    "id": 1, "name_ar": "سيارات", "name_en": null, "slug": "cars", "icon_url": null, "is_active": true,
    "children": [],
    "attributes": [
      { "id": 1, "key": "condition", "label_ar": "الحالة", "label_en": null, "type": "select", "options": [...] },
      { "id": 3, "key": "year", "label_ar": "سنة الصنع", "label_en": null, "type": "number", "options": [...] }
    ]
  }
}
```

`type` is one of `text|number|boolean|select|mselect` — it tells the client how to render the field, and the server how to validate it on `POST/PUT /ads` . `options` is only non-null for `select` / `mselect` . **Errors:** 404 if the category doesn't exist.

Ads

This is the core of the app. `status` is one of `pending|approved|rejected|expired|sold` — every new ad starts `pending`, and only an admin approving it (see Admin ► Ads) makes it visible on the public feed.

GET `/ads`

Auth: none. **Does:** The public browse/search feed. **Only** `status = approved` ads are ever returned here. Default order (no `sort` given, or `sort=newest`) is featured ads first, then newest first — the same as before `sort` existed, so omitting it doesn't change anything for existing clients. **Input (all query params optional):**

Param	Type	Does
<code>category_id</code>	int	exact match
<code>governorate_id</code>	int	exact match
<code>city_id</code>	int	exact match
<code>min_price</code> / <code>max_price</code>	number	inclusive range on <code>price</code>
<code>q</code>	string	matches against <code>title</code> or <code>description</code> (<code>LIKE %q%</code>)
<code>sort</code>	string	one of <code>newest</code> (default), <code>price_asc</code> , <code>price_desc</code> , <code>popular</code> (by <code>views_count</code> descending), <code>nearest</code> . Picking any value other than <code>newest</code> replaces the default featured-first ordering entirely — it doesn't layer on top of it.
<code>lat</code> / <code>lng</code>	number	required when <code>sort=nearest</code> (422 otherwise) — the caller's own coordinates, to sort by. Only ads with both <code>latitude</code> and <code>longitude</code> set participate in <code>nearest</code> ; ads without a location are excluded from the results entirely for that sort (not pushed to the end). Distance is an unscaled flat-earth approximation (fine for sort order, not an actual km/mi value) — not returned in the response.
any category-attribute key (e.g. <code>fuel_type</code> , <code>condition</code>)	string	matches ads whose dynamic attribute of that key equals the given value. If the key is defined on more than one category (e.g. <code>condition</code> exists on both "cars" and "motorcycles"), matches are OR'd across all of them unless <code>category_id</code> is also given to disambiguate. Only attributes with <code>is_filterable = true</code> are matchable this way; unrecognized params are silently ignored (not an error).

Output — 200 : paginated list of `AdResource` (20/page). Each item (note: `description` is **omitted** here — only present on the single-ad `show` endpoint):

```
{
  "id": 1, "title": "كيا سبورتاج 2019 للبيع", "slug": "kia-sportage-2019-test",
  "price": 15000, "currency": "USD", "status": "approved", "is_featured": false, "views_count": 0,
  "latitude": null, "longitude": null,
  "category": { "id": 1, "name_ar": "سيارات", ... },
  "governorate": { "id": 1, "name_ar": "دمشق", ... },
  "city": { "id": 1, "governorate_id": 1, "name_ar": "دمشق", ... },
  "images": [ { "id": 1, "url": "http://.../storage/ads/1/xyz.jpg", "is_cover": true } ],
  "created_at": "...", "published_at": "..."
}
```

GET /ads/{id}

Auth: none. **Does:** Full ad detail — the only endpoint that includes `description`, and the only one that includes `attributes` (the dynamic spec values) and the seller's `user` info. Also increments `views_count` by 1 on every call (no dedup — a refresh counts again). **Input:** `{id}` path param. **Output** — **200**: a single `AdResource`, plus:

```
"attributes": [ { "key": "condition", "label_ar": "الحالة", "value": "مستعملة" }, { "key": "year", "
```

Errors: **404** — either the ad genuinely doesn't exist, **or** it exists but isn't `approved` and the caller isn't looking at their own ad (deliberate: a pending/rejected ad is invisible to everyone except through `GET /my/ads`, even by direct ID).

GET /my/ads

Auth: required. **Does:** The caller's own ads, **any status** (pending/approved/rejected/etc.) — this is how a seller sees "my listing was rejected" or checks review status. **Input:** none. **Output** — **200**: paginated `AdResource` collection (20/page), same shape as the public feed items.

POST /ads

Auth: required. **Content-Type:** `multipart/form-data` (it carries files). **Does:** Creates a new ad as `status = pending`, awaiting admin review. Validates the dynamic `attributes[]` against the chosen category's `category_attributes` (required-ness, `select / multiselect` option membership, `number` type-checking). Resizes/compresses every uploaded image (max 1600×1600, re-encoded JPEG q80) before storing it — see `app/Services/ImageService.php`. **Input (form fields):**

Field	Required	Notes
<code>category_id</code>	yes	must exist
<code>governorate_id</code>	yes	must exist
<code>city_id</code>	yes	must exist
<code>title</code>	yes	string, max 150
<code>description</code>	yes	string, max 5000
<code>price</code>	no	numeric ≥ 0
<code>currency</code>	no	3-letter code, defaults to <code>USD</code>
<code>latitude / longitude</code>	no	numeric, valid coordinate range
<code>images[]</code>	yes	1–12 image files, each $\leq 5\text{MB}$ (<code>max:5120 KB</code>)
<code>attributes[<key>]</code>	conditionally	one field per dynamic spec on the category, e.g. <code>attributes[condition]=مستعملة&attributes[year]=2020</code> ; required keys depend on the category (see <code>GET /categories/{id}</code>)

Output — **201** : a single `AdResource` with `status: "pending"`, including `images` and `attributes` (freshly created). **Errors:**

- **401** if not authenticated.
- **422** — either standard field validation, or `{"errors": {"attributes": [{"قيمة", "مطلوب."}, {"\...\\" غير صالحة"}]}}` for dynamic-attribute problems.

PUT/PATCH `/ads/{id}`

Auth: required, **must be the ad's owner**. **Does:** Partial update (only send the fields you're changing) — `title`, `description`, `price`, and/or `attributes`. Photos are **not** managed here — see `POST / DELETE /ads/{id}/images` below. **Any edit resets the ad to `status = pending`** and clears `rejection_reason` / `reviewed_by` / `reviewed_at`, forcing a fresh admin review. **Input (JSON or form body, all optional):** `title`, `description`, `price`, `attributes` (object — same dynamic-attribute rules as create, but required-ness isn't re-enforced on a partial edit; an empty value for a key clears that attribute). **Output** — **200** : the updated `AdResource`. **Errors:** **403** if the caller isn't the owner; **422** on validation failure.

DELETE `/ads/{id}`

Auth: required, **must be the ad's owner**. **Does:** Soft-deletes the ad (recoverable in the DB, but gone from every endpoint immediately). **Input:** none. **Output** — **200** : `{"message": "تم حذف الإعلان"}`. **Errors:** **403** if not the owner.

POST `/ads/{id}/images`

Auth: required, **must be the ad's owner**. **Content-Type:** `multipart/form-data`. **Does:** Adds one or more photos to an ad that already exists, **appending** to its current gallery rather than replacing it (so adding one more photo doesn't require re-sending the rest). Same resize/compression as ad creation (max 1600×1600, JPEG q80). If the ad had no images at all (shouldn't normally happen, since creation requires ≥ 1), the first image added becomes the cover; otherwise the existing cover is left alone. Like any other edit, **resets the ad to `status = pending`** for re-review. **Input (form fields):**

Field	Required	Notes
<code>images[]</code>	yes	1 or more image files, each $\leq 5\text{MB}$. Total images on the ad (existing + new) cannot exceed 12.

Output — **200** : the updated `AdResource`, including the full `images` array. **Errors:**

- **403** if the caller isn't the owner.
- **422** — bad/undecodable file, or the 12-image cap would be exceeded.

DELETE `/ads/{id}/images/{imageId}`

Auth: required, **must be the ad's owner**. **Does:** Removes one photo from the ad. An ad must always keep **at least one** photo — removing the last one is rejected (delete the whole ad instead if that's the goal). If the removed photo

was the cover, the next photo by `sort_order` is automatically promoted to cover. Like any other edit, **resets the ad to status = pending** for re-review. **Input:** none. `{imageId}` is the `id` from an item in the ad's `images` array.

Output — 200 : the updated `AdResource` . **Errors:**

- 403 if the caller isn't the owner.
- 404 if `{imageId}` doesn't belong to this ad.
- 422 `{"message": "احذف الإعلان بالكامل إذا أردت"}` يجب أن يحتوي الإعلان على صورة واحدة على الأقل — احذف الإعلان بالكامل إذا أردت `if it's the ad's only remaining photo.`

Favorites

GET /favorites

Auth: required. **Does:** The caller's saved/bookmarked ads. **Output — 200 :** paginated `AdResource` collection (only ads the caller favorited).

POST /ads/{id}/favorite

Auth: required. **Does:** Saves the ad to the caller's favorites. Idempotent — calling it twice doesn't create a duplicate. **Output — 200 :** `{"message": "أُضيف إلى المفضلة"}` . **Errors:** 404 if the ad isn't `approved` — a pending/rejected ad was never shown to anyone, so it can't be favorited, same rule as `GET /ads/{id}` .

DELETE /ads/{id}/favorite

Auth: required. **Does:** Removes the ad from the caller's favorites (no-op, still `200` , if it wasn't favorited). **Output — 200 :** `{"message": "أُزيل من المفضلة"}` .

Chat

One conversation exists per `(ad, buyer)` pair — starting it twice for the same ad, as the same buyer, returns the same conversation rather than creating a second one.

GET /conversations

Auth: required. **Does:** All conversations the caller is part of, as either buyer or seller, newest-activity first. **Output — 200 :** paginated `ConversationResource` collection:

```
{ "id": 1, "ad": { ...AdResource }, "buyer": { ...UserResource }, "seller": { ...UserResource }, "last"
```

POST /ads/{id}/conversations

Auth: required. **Does:** Starts (or fetches the existing) conversation with the ad's owner about that ad. The caller becomes the `buyer` . **Output:** a `ConversationResource` . **201** the first time (new conversation), **200** on every

subsequent call for the same ad+buyer (Laravel auto-201s a POST whose result was freshly created — treat both as success). **Errors:**

- 404 if the ad isn't `approved` — same visibility rule as `GET /ads/{id}` and favoriting; you can't message about an ad you couldn't have seen.
- 422 `{"message": "لا يمكنك مراسلة نفسك"}` if you try to message yourself about your own ad.

`GET /conversations/{id}/messages`

Auth: required, **must be a participant** (buyer or seller on that conversation). **Does:** Full message history, oldest first (paginated 50/page — intended for polling, not one-shot infinite scroll). **Output** — 200 : paginated `MessageResource` collection:

```
{ "id": 1, "conversation_id": 1, "sender": { ...UserResource }, "body": "Hello, is this still availab
```

Errors: 403 if the caller isn't a participant.

`POST /conversations/{id}/messages`

Auth: required, **must be a participant**. **Does:** Sends a message, bumps the conversation's `last_message_at`, and **notifies the other participant** (see Notifications — `type: "new_message"`). **Input:** `{"body": "..."} (required, max 2000 chars). Output: a MessageResource ( 200 / 201 , same auto-201 note as above). Errors: 403 if not a participant; 422 if body missing/too long.`

Ratings

(`rater`, `rated`, `ad`) is a unique triple — rating the same person again (for the same `ad_id`, including both `null`) **updates** the existing rating rather than creating a second one, and the target's `rating_average` / `rating_count` are recomputed from scratch every time (see `app/Services/RatingService.php`).

`GET /users/{id}/ratings`

Auth: none — shown on a public seller profile. **Output** — 200 : paginated `RatingResource` collection:

```
{ "id": 2, "score": 4, "comment": "good seller", "rater": { ...UserResource }, "created_at": "..."} }
```

`POST /users/{id}/ratings`

Auth: required. **Does:** Rates another user, then immediately recomputes their `rating_average` / `rating_count`, then **notifies them** (`type: "new_rating"`). **No proof of an actual transaction/conversation is required** — a rating can reference any ad the rater could plausibly have seen (home feed or search), which by definition means an approved one. **Input:** `score` (required, integer 1–5), `comment` (optional, max 1000), `ad_id` (optional — must be an `approved` ad if given; not required to have any connection to the rater/rated beyond existing and being approved). **Output:** a `RatingResource` (200 / 201). **Errors:**

- 422 {"message": "لا يمكنك تقييم نفسك"} if {id} is your own user id.
- 422 if ad_id is given but the ad isn't approved .

Notifications

Backed by Laravel's standard database-notifications table (`Notifiable` trait on `User`). Every notification's `data` payload always has at least `type` and a human-readable Arabic `message` ; extra fields vary by type (see below).

type	Fired when	Extra data fields
ad_approved	Admin approves your ad	ad_id , title
ad_rejected	Admin rejects your ad	ad_id , title , reason
new_message	Someone messages you in a conversation	conversation_id , sender_id , sender_name , body
new_rating	Someone rates you	rating_id , rater_id , rater_name , score

GET /notifications

Auth: required. **Does:** The caller's notifications, newest first, read and unread mixed together (check `read_at` per item to distinguish). **Output — 200 :** paginated (30/page) raw notification rows (not a custom Resource — this is Laravel's default `DatabaseNotification` shape):

```
{
  "id": "141cc664-62f1-42cc-99c0-0f1a919f1ae6",
  "type": "App\\Notifications\\AdApproved",
  "notifiable_type": "App\\Models\\User", "notifiable_id": 3,
  "data": { "type": "ad_approved", "ad_id": 2, "title": "Test Car Ad", "message": "موافقة على إعلانك",
  "read_at": null, "created_at": "...", "updated_at": "..."}
}
```

Note `id` here is a UUID string, not an integer — pass it as-is to the two endpoints below.

POST /notifications/{id}/read

Auth: required. **Does:** Marks one notification read (`{id}` is the UUID from the list above). Silently succeeds even if the id doesn't exist or belongs to someone else (no-op, not an error — nothing to leak either way). **Output — 200 :** {"message": "تم التحديث"} .

POST /notifications/read-all

Auth: required. **Does:** Marks every one of the caller's unread notifications as read. **Output — 200 :** {"message": "تم التحديث"} .

Banners (sponsored, home-screen carousel)

Separate from `ads` entirely — no owner, no approval workflow, no in-app payment. A business pays the admin directly (e.g. a Sham Cash transfer arranged outside the app, same as cash) for a slot in the home-screen carousel for an agreed window, and the admin creates the banner here. See `docs/HOW_IT_WORKS.md` § Sponsored Banners for the full story.

GET `/banners`

Auth: none. **Does:** The banners currently inside their `starts_at` / `ends_at` window **and** not paused (`is_active = true`) — exactly what the home-screen carousel should render. Ordered by `sort_order` , then creation order. **Output** — `200` :

```
{
  "data": [
    { "id": 1, "title": "Pizza Palace", "image_url": "http://.../storage/banners/xyz.jpg", "link_url":
  ]
}
```

(Unpaginated — the full active set every time, like `/categories` and `/ad-packages` .)

GET `/admin/banners`

Auth: admin. **Does:** Every banner regardless of its window or `is_active` — the admin's own management list, not what the public carousel shows. **Output** — `200` : paginated (30/page) `BannerResource` collection.

POST `/admin/banners`

Auth: admin. **Content-Type:** `multipart/form-data` (carries the image). **Does:** Creates a banner. Resized/compressed the same way as ad photos (max 1600×1600, JPEG q80). **Input (form fields):**

Field	Required	Notes
<code>image</code>	yes	image file, ≤5MB
<code>title</code>	no	string, max 150 — admin-facing label, also returned to the client
<code>link_url</code>	no	must start with <code>https://</code> , <code>http://</code> , or <code>tel:</code> if given — whatever tapping the banner should open
<code>starts_at</code> / <code>ends_at</code>	yes	datetimes; <code>ends_at</code> must be ≥ <code>starts_at</code>
<code>sort_order</code>	no	integer, defaults to 0 — lower sorts first

Output — `201` : the new `BannerResource` . **Errors:** `422` on validation failure (including a rejected `link_url` scheme, e.g. `javascript:...`).

PUT/PATCH `/admin/banners/{id}`

Auth: admin. **Does:** Partial update of any of `title`, `link_url`, `starts_at`, `ends_at`, `sort_order`, `is_active`. Flipping `is_active` to `false` is how an admin pauses a banner early **without** recalculating its dates. Does **not** support replacing the image — delete and recreate the banner instead. **Output — 200**: the updated `BannerResource`.

DELETE `/admin/banners/{id}`

Does: Deletes the banner and its stored image file. **Output — 200**: `{"message": "تم الحذف"}`.

Payments (Sham Cash) — partially implemented

GET `/ad-packages`

Auth: none. **Does:** The active featured-ad packages (15/30/60-day boosts), cheapest/shortest first. **Output — 200**: `{"data": [ { "id": 1, "name_ar": "...", "duration_days": 15, "price": 5.0, "currency": "USD" }, ... ]}` (unpaginated — the full active catalog every time).

POST `/ads/{id}/checkout` — **stub**

Auth: required, must be the ad's owner. **Input:** `{"ad_package_id": <id>}`. **Current behavior:** creates a `pending` `Payment` row, then **always returns 501** `{"message": "Sham Cash integration pending – see app/Services/ShamCash/ShamCashClient.php"}`. Waiting on real Sham Cash API docs — see `app/Services/ShamCash/ShamCashClient.php`.

POST `/payments/shamcash/webhook` — **stub**

Auth: none (would be HMAC-signature-verified once implemented). **Current behavior: always returns 501** `{"message": "Not implemented yet."}`.

Admin (requires the `admin` role in addition to auth — 403 `{"message": "Admins only."}` otherwise)

Admin ▶ Categories

GET `/admin/categories`

Does: Every category (regardless of `is_active`), each with `children` and `attributes` loaded — the full management view. **Output — 200**: `{"data": [ ...CategoryResource ]}` — **unpaginated**, full set every time.

POST `/admin/categories`

Input: `parent_id` (optional), `name_ar` (required), `name_en` (optional), `sort_order` (optional). `slug` is auto-generated from `name_en / name_ar` + a random suffix. **Output — 201**: the new `CategoryResource`.

PUT/PATCH /admin/categories/{id}

Input (all optional): `name_ar`, `name_en`, `is_active`, `sort_order`. **Output — 200** : the updated `CategoryResource`.

DELETE /admin/categories/{id}

Output — 200 : `{"message": "تم الحذف"}`. **Errors:** 409 `{"message": "لا يمكن حذف هذه الفئة لوجود إعلانات مرتبطة بها أو بإحدى فئاتها الفرعية"}` if any ad (directly or via a subcategory) still references this category.

POST /admin/categories/{id}/attributes

Does: Adds one dynamic spec field to the category. **Input:** `key` (required, e.g. `fuel_type`), `label_ar` (required), `label_en` (optional), `type` (required, one of `text|number|boolean|select|multiselect`), `options` (array, required in practice for `select / multiselect`), `is_required` (bool), `is_filterable` (bool — whether it's usable as a `GET /ads` query filter). **Output — 201** : the raw `CategoryAttribute` row as JSON (not wrapped in `{"data": ...}`) — this one endpoint returns the model directly).

Admin ▸ Users

GET /admin/users?q=

Does: Search/list users by name or phone (`q` optional — omit for the full paginated list). **Output — 200** : paginated `UserResource` collection (30/page).

POST /admin/users/{id}/ban

Does: Sets `is_banned = true` and immediately revokes every one of the user's existing Sanctum tokens, so a banned user is logged out everywhere right away, not just blocked from their *next* login. Admins can't ban other admins (including themselves) — there's no way to un-admin someone through this endpoint. **Output — 200** : the updated `UserResource` (`is_banned: true`). **Errors:** 422 if `{id}` has the `admin` role.

POST /admin/users/{id}/unban

Output — 200 : the updated `UserResource` (`is_banned: false`).

Admin ▸ Ads

GET /admin/ads?status=

Does: The moderation queue. `status` optional — filters to `pending / approved / rejected /etc.`, or omit for everything. **Output — 200** : paginated `AdResource` collection (30/page), each including `user` and `category`.

POST /admin/ads/{id}/approve

Does: Sets `status = approved`, stamps `published_at / reviewed_by / reviewed_at`, and **notifies the owner** (`type: "ad_approved"`). **Output — 200** : the updated `AdResource`.

POST /admin/ads/{id}/reject

Does: Sets `status = rejected`, stores the reason, stamps `reviewed_by / reviewed_at`, and **notifies the owner** (`type: "ad_rejected"`). **Input:** `{"reason": "..."} (required, max 500)`. **Output — 200 :** the updated `AdResource` . **Errors:** 422 if `reason` is missing.